DAY 09 OF 16 · 3D & 6DOF STREAMING

From geometry to meaning

Teach the computer what the points are, ground, walls, trees, objects, so a stream can send the important things first.

PCA features | RANSAC | DBSCAN | Evaluation | Semantics for streaming

Book: Ch. 6–7, 9, 12 · Code: day09_scene_understanding/

BOOK CH. 6–7 · FEATURES

Each point’s neighbourhood has a shape

PCA

Take the 50 nearest neighbours; the eigenvalues $\lambda 1 \geq \lambda 2 \geq \lambda 3$ of their covariance describe the local shape.

Planarity

$(\lambda 2 - \lambda 3) / \lambda 1$: high on walls, roofs and ground.

Sphericity

$\lambda 3 / \lambda 1$: high where points scatter in all directions, like foliage.

Verticality

From the normal (smallest eigenvector): ≈ 1 on walls and trunks, ≈ 0 on the ground.

BOOK CH. 9 · RANSAC

RANSAC finds a plane by voting

01

Sample

Pick 3 random points.

02

Fit

They define a plane.

03

Count

How many points lie within 5 cm?

04

Keep the best

Repeat 500 times; keep the plane with most inliers.

Lab result: the ground plane with 98% precision and 99.8% recall against the true labels.

BOOK CH. 12 · DBSCAN

DBSCAN groups what touches

Rule

A point with at least 10 neighbours within 0.5 m is a core point; connected core points form a cluster.

01

Strengths

No need to say how many clusters; finds any shape; labels noise.

02

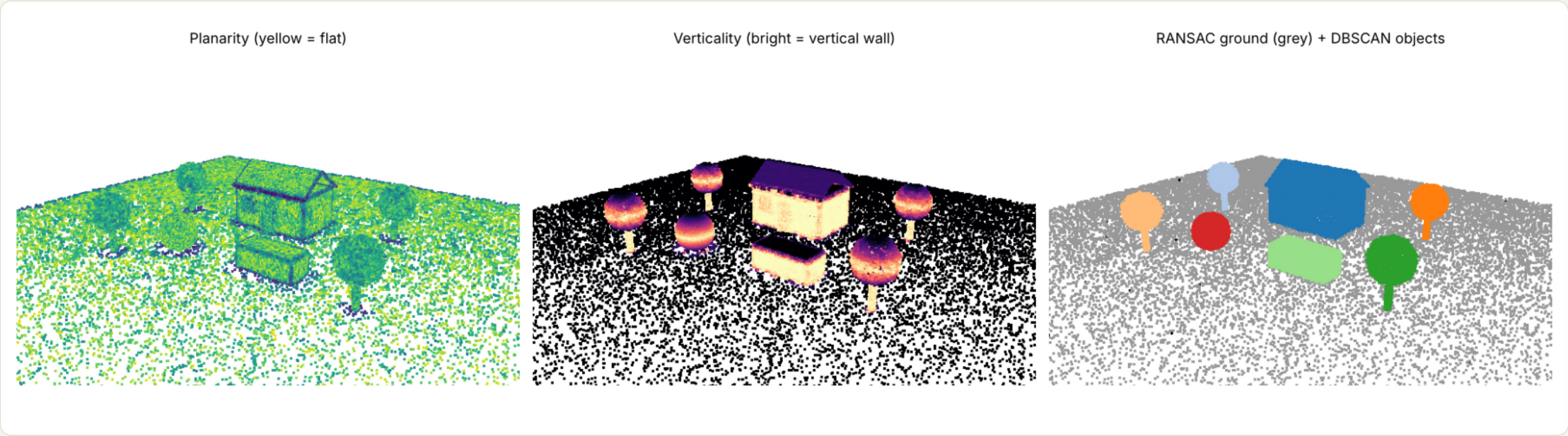
Lab result

7 objects: the house, 4 trees, the car and the sphere, plus 60 noise points.

03

LAB RESULT

Features, ground and objects



Left: planarity. Middle: verticality. Right: RANSAC ground (grey) and DBSCAN objects (colours).

BACK TO STREAMING

Semantics makes streaming smarter

Priority

Send people, faces and objects of interest first; ground and sky later.

01

Quality budget

Spend bits where viewers look: objects, not the floor.

02

Interaction

Clickable objects, occlusion with real objects in AR, physics.

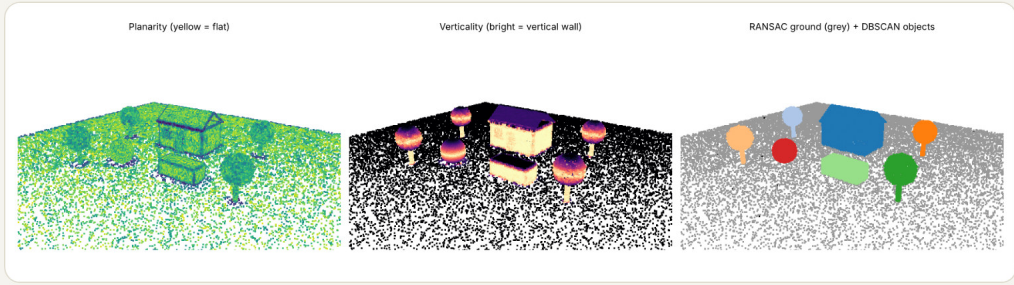
03

HANDS-ON LAB · COPY, RUN, LOOK

Segment the street scene

RUN IT

```
cd day09_scene_understanding
python segment_scene.py
```



→ Read the feature table per class.

→ Read the object list from DBSCAN.

→ Change eps to 0.2 and 1.5.

Three things to remember

1. PCA of a neighbourhood gives planarity, sphericity and verticality per point.
2. RANSAC fits shapes by random voting; DBSCAN groups points that touch.
3. Labels let a stream spend its bytes on what matters.

BEFORE NEXT SESSION

Lab L5 question 1. Next: surfaces, mesh LoDs and Gaussian splats.

DAY 10 OF 16 · 3D & 6DOF STREAMING

Surfaces, mesh LoDs & splats

From points to watertight meshes, from heavy meshes to light ones, and why 3D Gaussian splats look so real.

Poisson meshing | Mesh LoDs | NeRF | Gaussian splats | What to stream?

Book: Ch. 10, 18 · Code: day10_surfaces_neural/

BOOK CH. 10

Poisson reconstruction: a smooth surface through the points

Input

Points with consistent normals (Day 8).

01

Idea

Find an inside/outside function whose gradient matches the normals; the surface is its zero level.

02

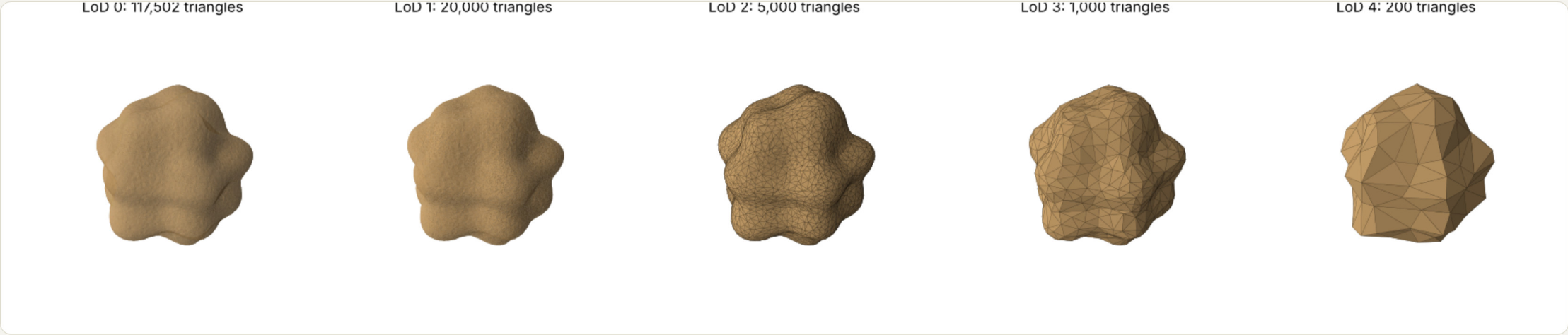
Depth

Octree depth controls detail: depth 8 gave 117k triangles from 30k points.

03

BOOK CH. 10 · LEVELS OF DETAIL

Mesh LoDs: 117k → 200 triangles



4.3 MB, 753 KB, 189 KB, 38 KB, 8 KB. From far away, LoD 3 looks the same as LoD 0.

BOOK CH. 18

Radiance fields: from geometry to appearance

Photogrammetry

Points and textured meshes; struggles with glass, reflections, hair.

01

NeRF (2020)

A neural network maps (position, direction) → colour and density. Photo-real, but slow to train and render.

02

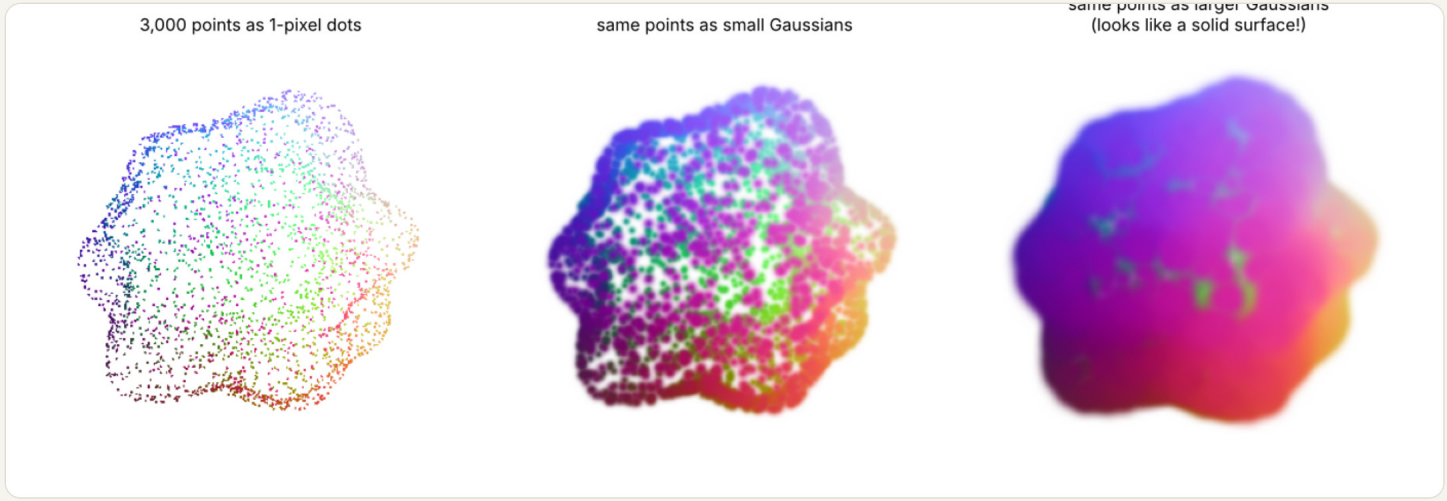
3D Gaussian Splatting (2023)

Millions of coloured, oriented, semi-transparent ellipsoids. Photo-real AND real-time.

03

LAB PREVIEW

Why splats look solid



- Back-to-front sort + alpha blending
- Size shrinks with distance
- Real 3DGS: 59 floats ≈ 236 B per splat
- vs 15 B per coloured point

Same 3,000 points. Drawn as soft, blended Gaussians, the gaps disappear and the surface looks continuous.

CHOOSING WHAT TO STREAM

Points, meshes or splats?

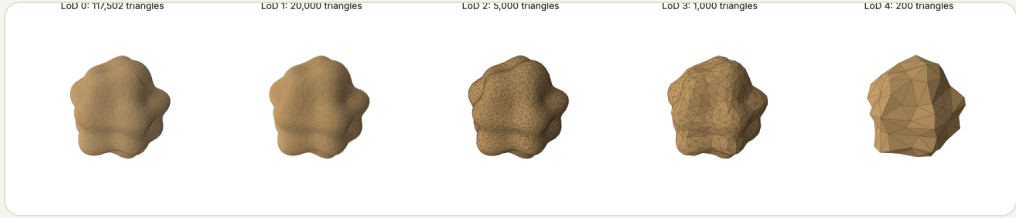
	Points	Meshes	Gaussian splats
Bytes per primitive	≈ 15 B	≈ 30–50 B / triangle	≈ 236 B (uncompressed)
Look	Holes up close	Clean surfaces	Photo-real, soft
Progressive LoD	Easy (octree)	Possible (decimation)	Research topic
Standards	MPEG G-PCC / V-PCC	glTF, 3D Tiles	Emerging

HANDS-ON LAB · COPY, RUN, LOOK

Mesh, simplify, splat

RUN IT

```
cd day10_surfaces_neural
python mesh_lod_and_splats.py
```



→ Read the LoD table: triangles vs file size.

→ Open outputs/rock_lod*.ply.

→ Compare the three panels of gaussian_splats.png.

Three things to remember

1. Poisson turns oriented points into a watertight mesh; depth sets detail.
2. Mesh LoDs shrink 117k triangles to 200 for distant views.
3. Gaussian splats look photo-real but cost ~16× more bytes per primitive than points.

BEFORE NEXT SESSION

Lab L5 and mini-project P2 (LoD builder). Next: the second half, viewing and streaming.

DAY 11 OF 16 · 3D & 6DOF STREAMING

Rendering & the 6DoF viewer

How a viewer turns a 6DoF pose into a picture 60 times a second, and why it only ever needs part of the scene.

Render pipeline · View frustum · Culling · A browser 6DoF viewer · Pose traces

Book: Ch. 5, 17 · Code: day11_rendering_6dof_viewing/

RENDERING

From 3D points to pixels, every frame

01

Pose

Read the 6DoF pose from mouse, keyboard or headset.

02

View matrix

Inverse pose: world → camera (Day 2).

03

Projection

Pinhole K: camera → image (Day 6).

04

Cull & draw

Skip what is outside the view; z-buffer keeps the nearest.

CULLING

The view frustum: a pyramid of what you can see

Six planes

Near, far, left, right, top, bottom. Set by the field of view and aspect ratio.

01

Test

A point is visible if it is in front of all six planes. For a tile, test its bounding sphere.

02

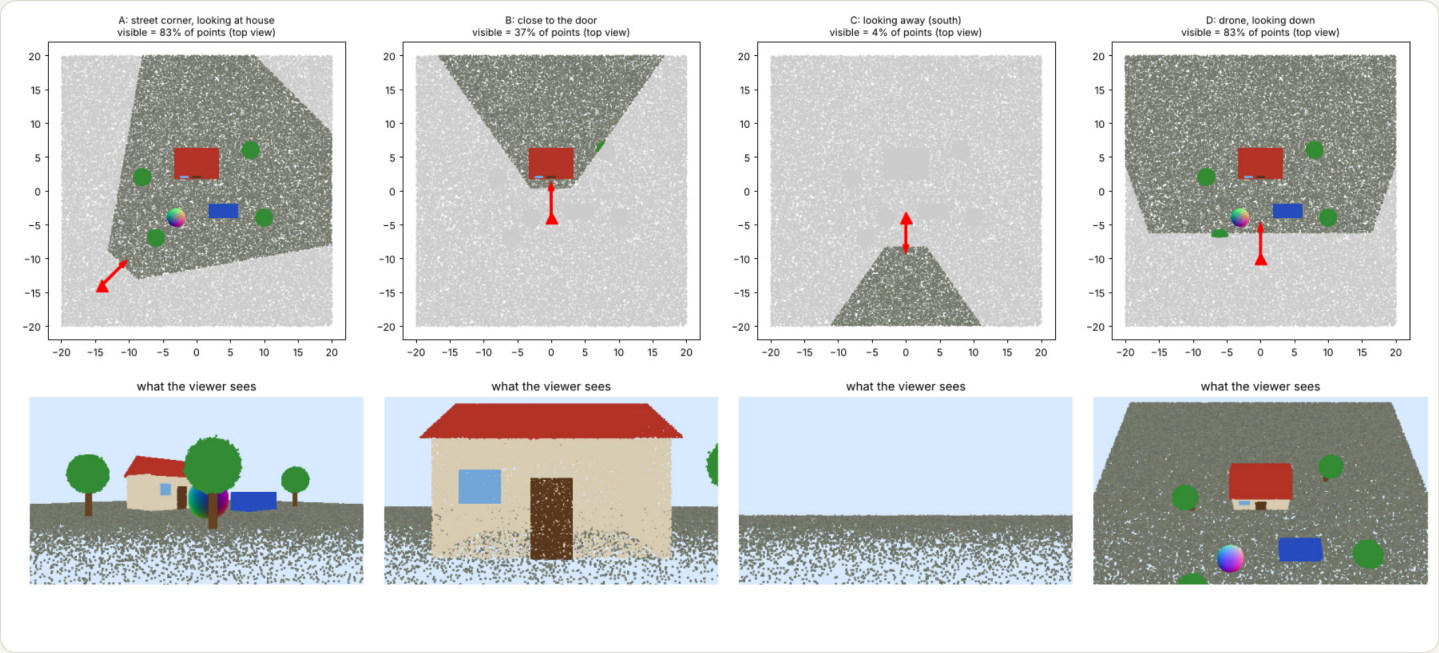
Payoff

Looking away from the town, only 4% of the points are visible.

03

LAB RESULT

What each pose actually sees



Grey = outside the frustum. The visible share swings from 4% to 83% with the pose alone.

BROWSER DEMO

A real 6DoF point cloud viewer

- 250k points rendered at interactive rates
- ‘Visible in frustum’ updates live
- ● Record → pose_trace.csv (for Day 16)
- Same controls in every demo of the course

Fly with WASD, drag to look, Q/E to roll. The HUD shows the pose as position, Euler angles and quaternion.

HANDS-ON LAB · COPY, RUN, LOOK

Measure the frustum, then fly

RUN IT

```
cd day11_rendering_6dof_viewing
python frustum_culling.py
cd viewer6dof
python -m http.server 8000
# open http://localhost:8000
```

→ Open outputs/frustum.png.

→ Fly around; watch ‘Visible in frustum’.

→ Record a 30 s pose trace and keep the CSV.

Three things to remember

1. Render loop: pose → view matrix → projection → cull → draw, every frame.
2. The view frustum shows that a viewer sees only a fraction of the scene.
3. A pose trace records 6DoF behaviour: the input for prediction and evaluation.

BEFORE NEXT SESSION

Lab L6 question 1 with your pose traces. Next: what streaming means in numbers.

DAY 12 OF 16 · 3D & 6DOF STREAMING

Streaming fundamentals

Bitrate, bandwidth, buffers, stalls and latency: the vocabulary of every streaming system, from Netflix to volumetric video.

Data rates | Segments & manifests | Buffer & stalls | Latency | 6DoF specifics

Book: beyond the book · Code: day12_streaming_fundamentals/

HOW BIG IS IT?

Raw data rates of different media

Content	Raw rate	Typical compressed
HD video 1080p, 30 fps	≈ 1.5 Gbit/s	≈ 5 Mbit/s
360° video 8K, 30 fps	≈ 21 Gbit/s	tens of Mbit/s
Volumetric 100k pts × 15 B, 30 fps	≈ 360 Mbit/s	→ Day 13
Volumetric 1M pts × 15 B, 30 fps	≈ 3.6 Gbit/s	→ Day 13

HOW STREAMING WORKS

Segments, a manifest and a client that decides

01

Encode

Cut content into short segments (1–4 s), each at several qualities.

02

Manifest

A file listing segments, qualities and sizes (DASH .mpd, HLS .m3u8).

03

Client

Downloads segments in order over plain HTTP.

04

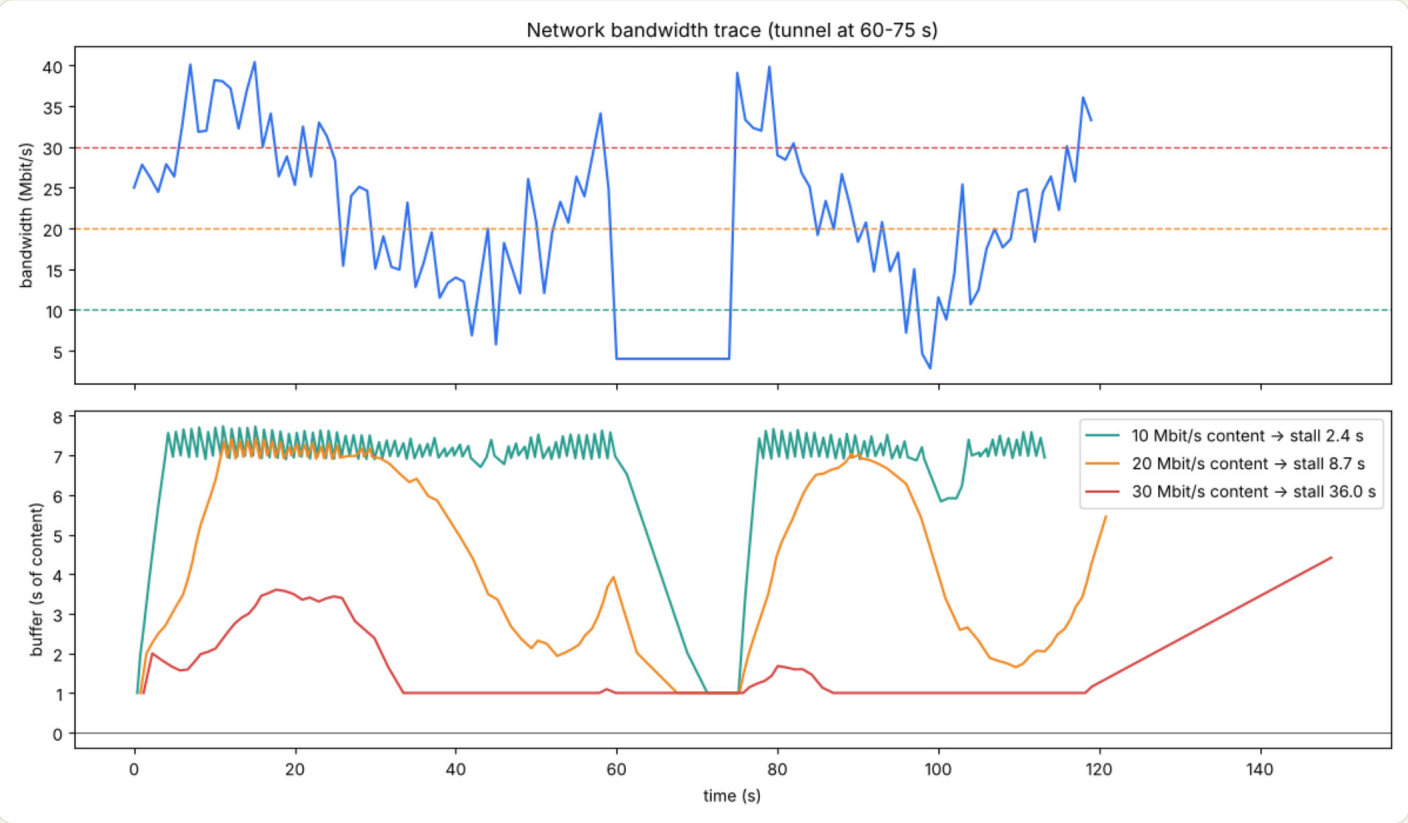
Adapt

Picks the quality of each next segment from throughput and buffer.

6DoF adds a new dimension to the choice: not only WHICH quality, but WHICH part of the scene.

LAB RESULT

The buffer: your safety margin against bad networks



Content at 10 Mbit/s stalls 2.4 s; at 20 Mbit/s 8.7 s; at 30 Mbit/s 36 s. The tunnel at 60–75 s drains every buffer.

6DOF SPECIFICS

Motion-to-photon: from head move to updated image

```
sensor 2 ms + pose 1 ms + render 11 ms + display 5 ms
≈ 19 ms
```

Target

Below about 20 ms, or users notice lag and may feel sick.

Network RTT

A 40 ms round trip alone breaks the budget.

Consequence

Render locally; stream the CONTENT ahead of time, not the pixels.

HANDS-ON LAB · COPY, RUN, LOOK

Simulate a stream with a buffer

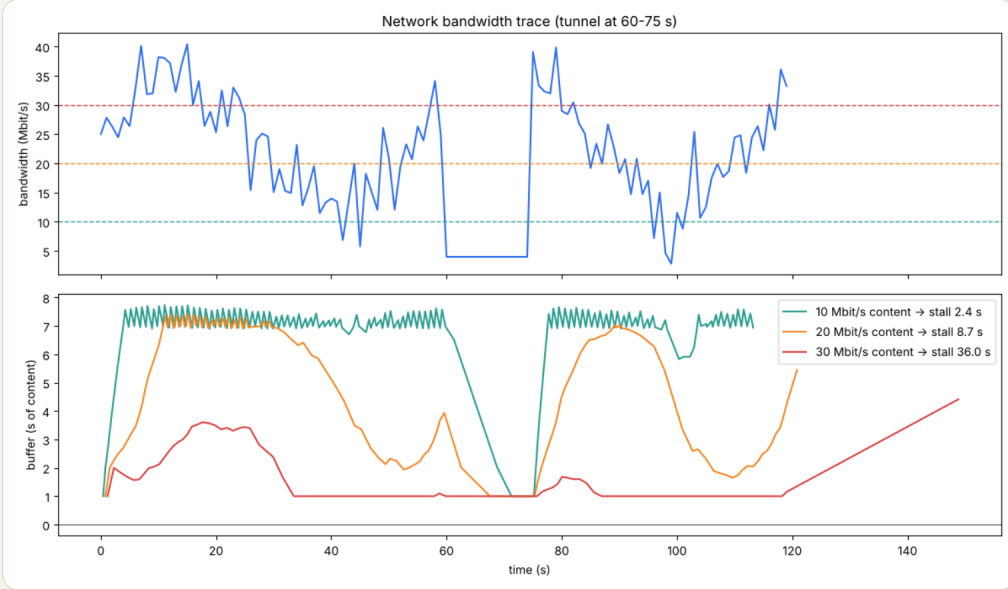
RUN IT

```
cd day12_streaming_fundamentals
python bitrate_buffer_latency.py
```

→ Part A: raw rate table.

→ Part B: buffer_simulation.png and stall times.

→ Part C: the latency budget.



Three things to remember

1. Raw 3D is hundreds of Mbit/s; networks give tens and fluctuate.
2. Streaming = segments + manifest + a client that adapts. The buffer absorbs dips; empty buffer = stall.
3. 6DoF needs < 20 ms motion-to-photon, so content is streamed ahead and rendered locally.

BEFORE NEXT SESSION

Lab L6 and mini-project P2 due. Next: compressing point clouds.

DAY 13 OF 16 · 3D & 6DOF STREAMING

Compressing point clouds

Quantise, code the octree, or turn it into video: the three ideas behind Draco and MPEG’s point cloud codecs.

Quantisation | D1 PSNR | Octree coding | Rate–distortion | V-PCC idea

Book: Ch. 4, 17 + MPEG · Code: day13_compression/

IDEA 1

Quantisation: snap points to a grid

How

Divide the bounding box into 2^b steps per axis; store integers.

01

Effect

10 bits on a 6 m house = 6 mm steps; nearby points merge.

02

Measure

D1 PSNR: point-to-point error relative to the box diagonal. 10 bits → 69 dB, 6 bits → 45 dB.

03

IDEA 2 · G-PCC

Octree occupancy coding: 1 byte per node

$$\text{byte} = \sum 2^{c} \text{ for each occupied child } c \in \{0 \ldots 7\}$$

Breadth-first

Root byte, then each child's byte, level by level.

Empty space is free

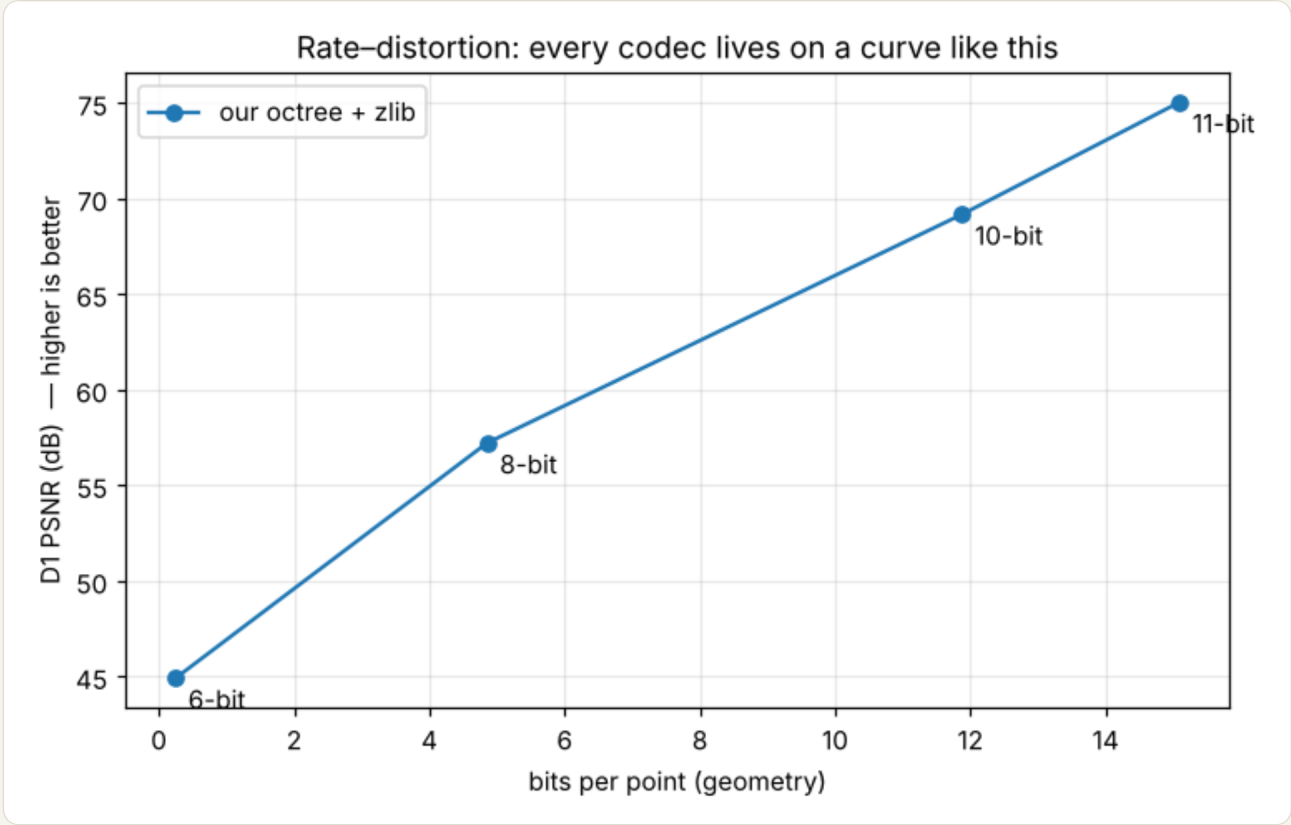
An empty octant is one 0 bit, its whole subtree is skipped.

Then entropy-code

G-PCC uses context arithmetic coding; we use zlib.

LAB RESULT

Every codec lives on a rate–distortion curve

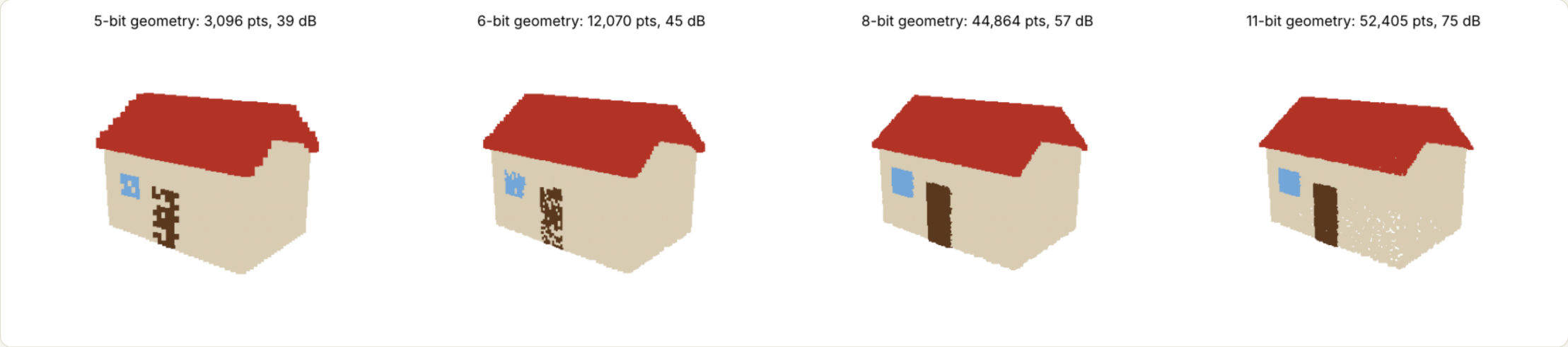


- Left = fewer bits, up = better quality
- Each dot = one quantisation level
- Codecs compete on the whole curve
- Colour coded separately (attributes)

Our octree + zlib: 0.25 bits/point at 45 dB up to 15 bits/point at 75 dB. Install DracoPy to add Draco.

WHAT THE NUMBERS LOOK LIKE

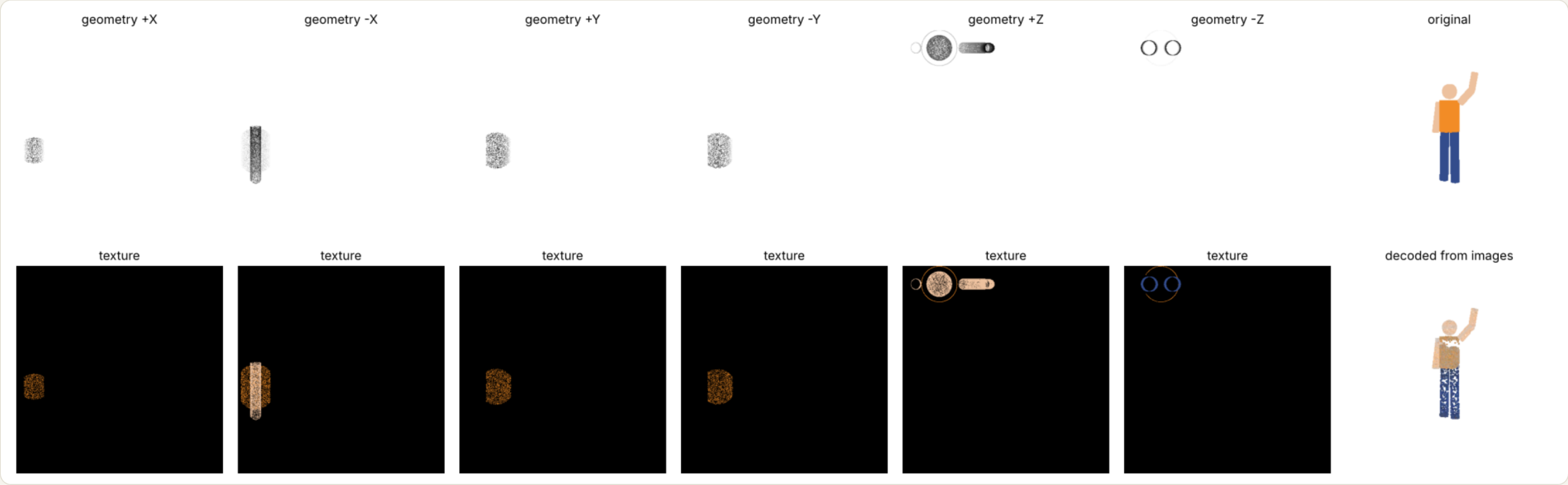
Quantisation you can see



5 bits: blocky. 8 bits: fine for distant viewing. 11 bits: indistinguishable. Distance decides how many bits you need.

IDEA 3 · V-PCC

Turn 3D into images, use a video codec



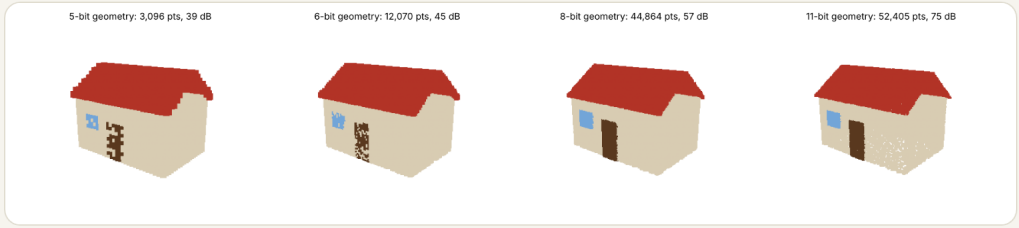
6 depth images + 6 colour images → PNG/JPEG. 1.8 mm mean error; points hidden within a projection are lost.

HANDS-ON LAB · COPY, RUN, LOOK

Write a codec, then draw its curve

RUN IT

```
cd day13_compression
python compress_pointcloud.py
python vpcc_projection_toy.py
pip install DracoPy   # optional
```



→ Read quantisation and octree tables.

→ Open rate_distortion.png.

→ Open vpcc_toy.png and compare original vs decoded.

Three things to remember

1. Quantisation trades precision for bits; D1 PSNR measures the damage.
2. Octree occupancy coding (G-PCC) sends 1 byte per node and nothing for empty space.
3. V-PCC turns 3D into 2D videos to reuse video codecs and hardware.

BEFORE NEXT SESSION

Lab L7. Next: send only what the viewer can see.

DAY 14 OF 16 · 3D & 6DOF STREAMING

Viewport-adaptive 6DoF streaming

Tiles × levels of detail × the viewer’s pose: send the right detail, for the right part of the scene, at the right time.

Tiles | LoD per tile | Visibility & distance | Budgets | 4 strategies compared

Book: beyond the book · Code: day14_viewport_adaptive/

THE RECIPE

Four decisions per update

01

Tile

Cut the scene into cells (here 5 m × 5 m).

02

LoD

Store each tile at 5 detail levels.

03

Select

Outside frustum → skip. Far → coarse. Near → fine.

04

Budget

Never exceed the bytes the network allows.

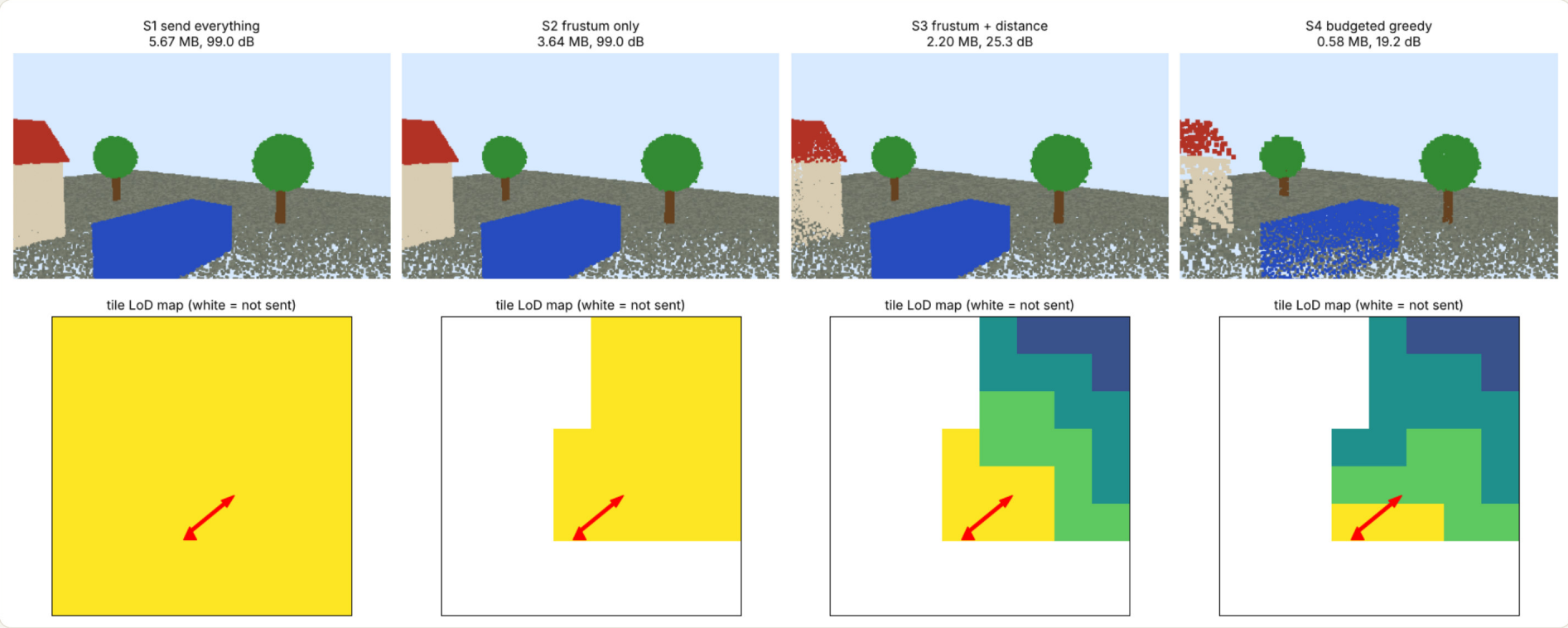
LAB RESULT

Four strategies along the same camera path

Strategy	MB / update	vs. everything	Image PSNR
S1 Send everything, full detail	5.67	100%	reference
S2 Frustum only, full detail	3.05	54%	identical
S3 Frustum + distance LoD	1.24	22%	34.4 dB
S4 S3 within a 0.6 MB budget	0.55	10%	25.8 dB

WHAT THE CLIENT ASKS FOR

Tile LoD maps for one pose



Bottom row: white tiles are not sent; brighter = finer LoD. The red arrow is the viewer.

OPEN PROBLEMS

What makes this hard in practice

Latency

By the time a tile arrives the user has turned. → predict the pose (Day 16).

01

Quality metric

Distance is a proxy; screen-space error or perceptual metrics are better.

02

Fairness of bits

Semantics (Day 9): a face matters more than a wall.

03

HANDS-ON LAB · COPY, RUN, LOOK

Compare four streaming strategies

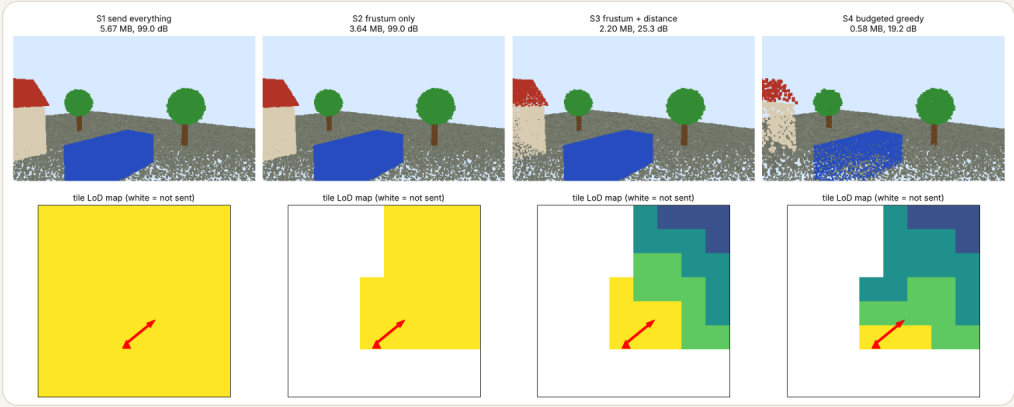
RUN IT

```
cd day14_viewport_adaptive
python viewport_adaptive.py
```

→ Read the strategy table.

→ Open strategies.png and tile_lod_maps.png.

→ Change BUDGET and re-run.



Three things to remember

1. Tiles × LoDs + the pose = what to request.
2. Culling alone halves the data; distance-based LoD cuts it to about a fifth.
3. A byte budget plus a benefit-per-byte rule keeps the stream inside the network.

BEFORE NEXT SESSION

Lab L8 question 1 and mini-project P3. Next: run it live over a network.

DAY 15 OF 16 · 3D & 6DOF STREAMING

A live 6DoF streaming system

A Python server, a browser client, a simulated network: walk through a streamed world and watch every byte.

Architecture | Manifest & chunks | Client logic | Throughput | Experiments

Book: beyond the book · Code: day15_live_6dof_streaming/

ARCHITECTURE

A dumb server and a smart client

01

Package

Server tiles the scene, makes 5 LoDs per tile, encodes 9 B/ point.

02

Manifest

manifest.json: tiles, bounding spheres, LoD sizes.

03

Link

Every chunk passes through a shared, rate-limited link.

04

Client

Every 150 ms: frustum, distance, throughput → next requests.

CLIENT/INDEX.HTML

What the client decides every 150 ms

Target LoD

Not visible → nothing new. Visible → LoD from distance, scaled by measured throughput.

Progressive

Ask for the NEXT level only: coarse first, then refine.

Priority

Visible first, then lowest current quality, then nearest. At most 4 requests in flight.

Throughput

Bytes received ÷ time the link was busy, smoothed (EWMA).

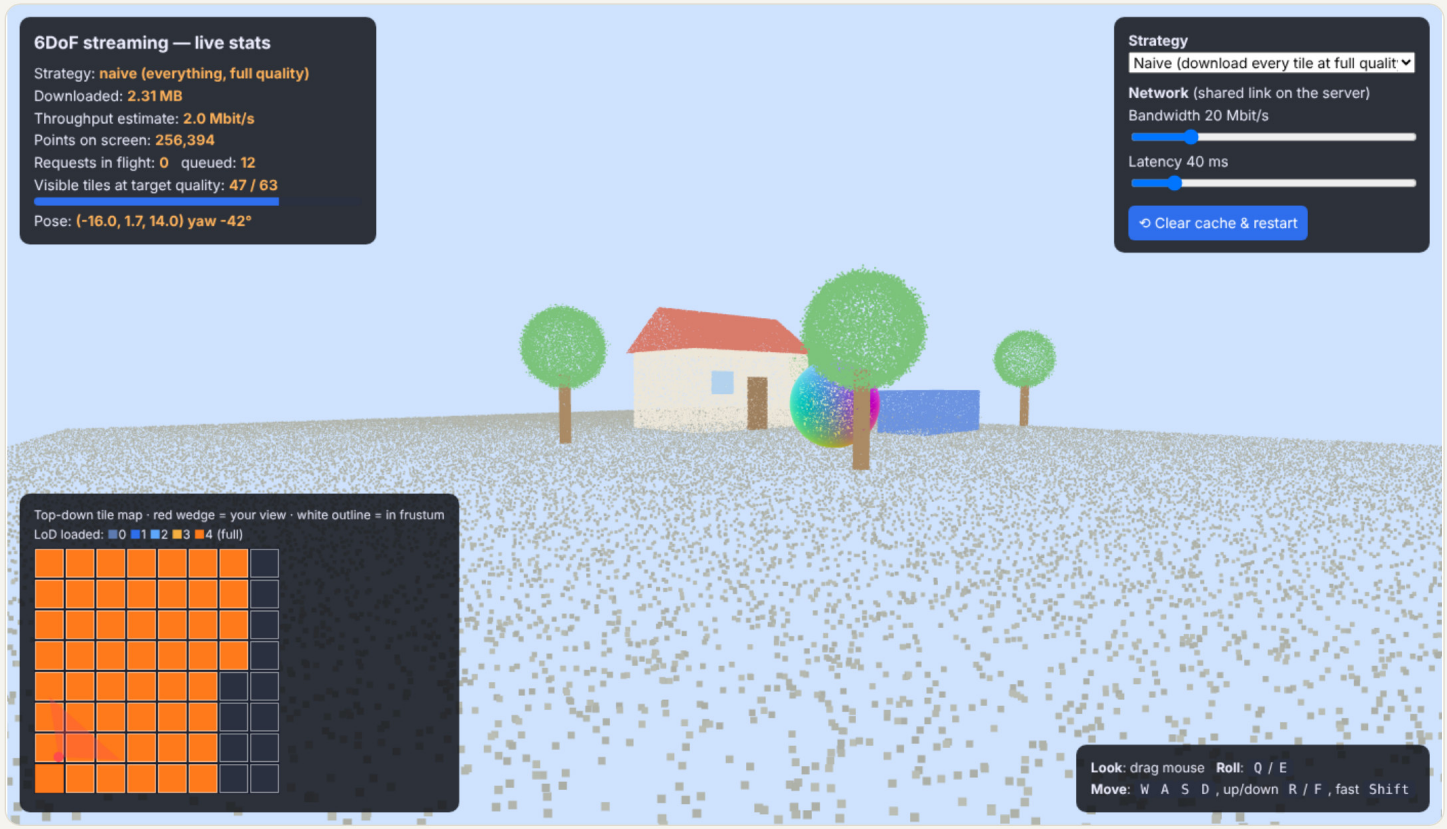
LIVE DEMO

Viewport-adaptive: quality where you look

Bottom-left map: tile colours = loaded LoD, red wedge = your view. Coarse first, refined near you.

COMPARE

Naive: download everything at full quality



Naive mode ignores the view: it fetches tiles in a fixed order, so at low bandwidth you wait for tiles you cannot even see.

EXPERIMENTS

Try these in class

Experiment	Set	Watch
Slow network	2 Mbit/s	Time until visible tiles reach target
High latency	300 ms	Turning fast: black gaps, late tiles
More parallelism	MAX_INFLIGHT 1 vs 16	Throughput estimate and fairness
Strategy race	adaptive vs naive	MB downloaded for the same view

HANDS-ON LAB · COPY, RUN, LOOK

Run the server and fly through the stream

RUN IT

```
cd day15_live_6dof_streaming
python server.py
# open http://localhost:8000
python server.py --kbps 3000   # slow link
```

→ Watch the tile map while you walk and turn.

→ Switch to naive and compare at 2 Mbit/s.

→ Read the server log: which tiles and LoDs?

Three things to remember

1. Server = manifest + chunks + a network; client = all the decisions.
2. The client’s loop: frustum, distance, throughput → progressive requests by priority.
3. Good 6DoF streaming shows something useful immediately and refines where you look.

BEFORE NEXT SESSION

Lab L8. Form capstone teams. Next: volumetric video, ABR and QoE.

DAY 16 OF 16 · 3D & 6DOF STREAMING

Volumetric video, ABR & QoE

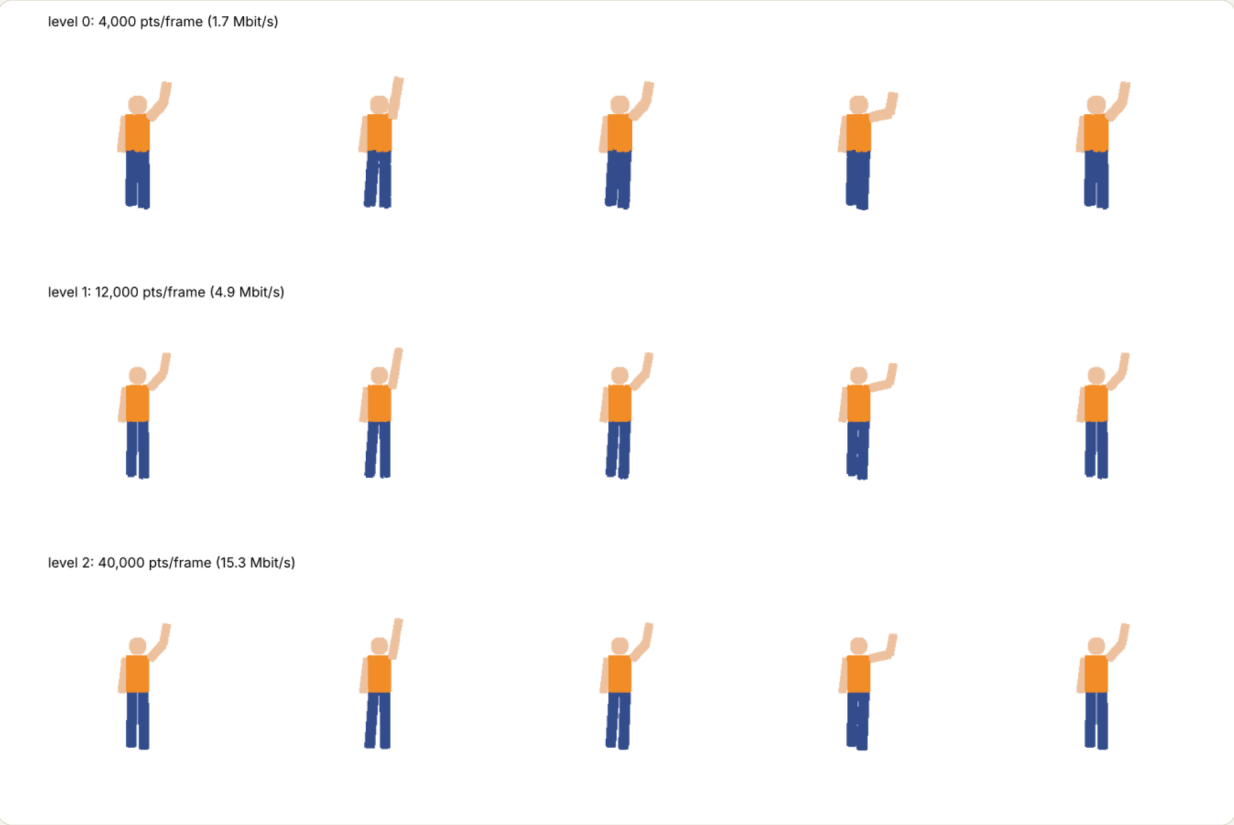
3D that moves: streaming a waving person you can walk around, choosing quality on the fly, and predicting where the viewer will look.

Volumetric video | Bitrate ladder | ABR algorithms | QoE | Pose prediction | Capstone

Book: Ch. 18 + streaming research · Code: day16_volumetric_video_qoe/

VOLUMETRIC VIDEO

A sequence of point clouds you can walk around



- Frame = point cloud; 15 fps
- Ladder ≈ 1.8 / 4.9 / 15 Mbit/s
- Segments of 1 s, like DASH
- MPEG standardises this as V-PCC content

3 quality levels: 4k, 12k and 40k points per frame. Each 1-second segment is encoded at all three.

ADAPTIVE BITRATE

Three ways to choose the next segment's quality

Fixed

Always the best level. Great until the network dips.

01

Throughput-based

Highest level below 80% of recent measured throughput. Reacts to the network; can be over-cautious.

02

Buffer-based (BBA)

Low quality when the buffer is nearly empty, high when it is full. Uses the buffer as the signal (Huang et al., SIGCOMM 2014).

03

QUALITY OF EXPERIENCE

One number to compare algorithms

$$QoE = \Sigma \text{ bitrate} - \mu \cdot \text{stall seconds} - \Sigma \left|\text{bitrate change}\right|$$

Bitrate

Higher quality is better.

μ · stall

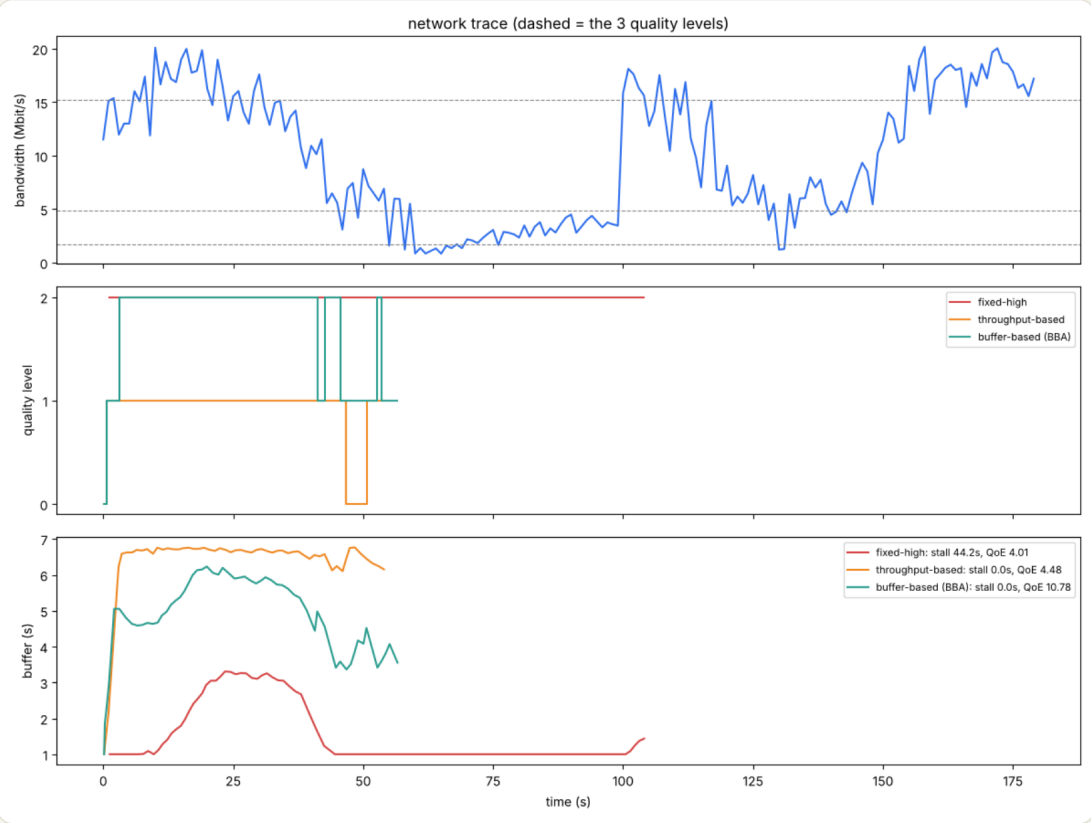
Stalls hurt most; μ = top bitrate, so 1 s of stall costs 1 s of top quality.

Switches

Frequent quality jumps are annoying.

LAB RESULT

Buffer-based ABR wins on this trace

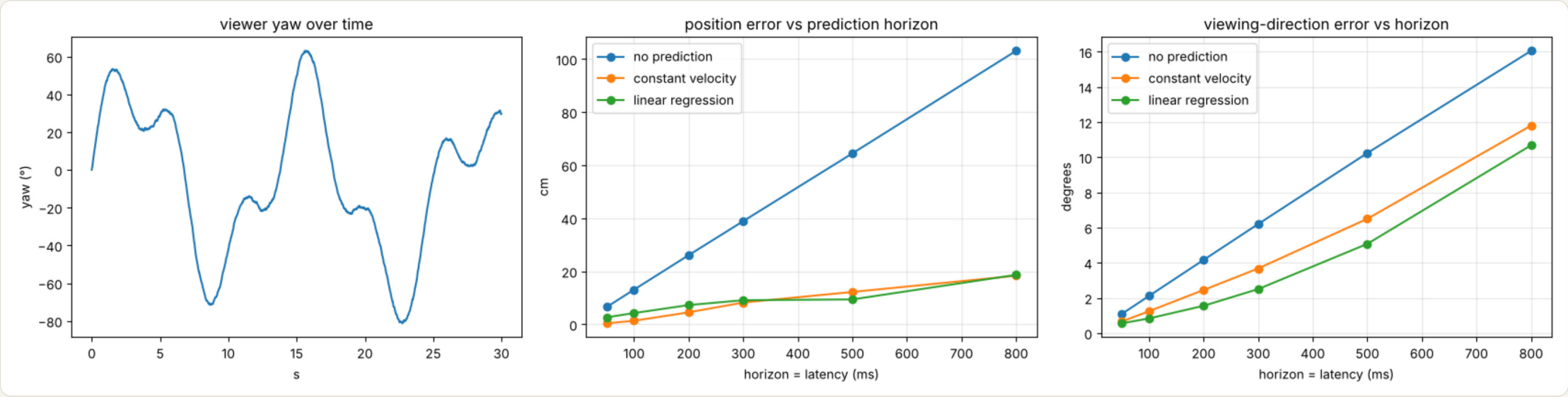


- Fixed-high: QoE 4.0, 44 s stalled
- Throughput: QoE 4.5, cautious
- Buffer-based: QoE 10.8
- Ranking depends on the network trace

Fixed-high stalls 44 s. Throughput-based never stalls but stays low. BBA keeps a buffer and quality high.

POSE PREDICTION

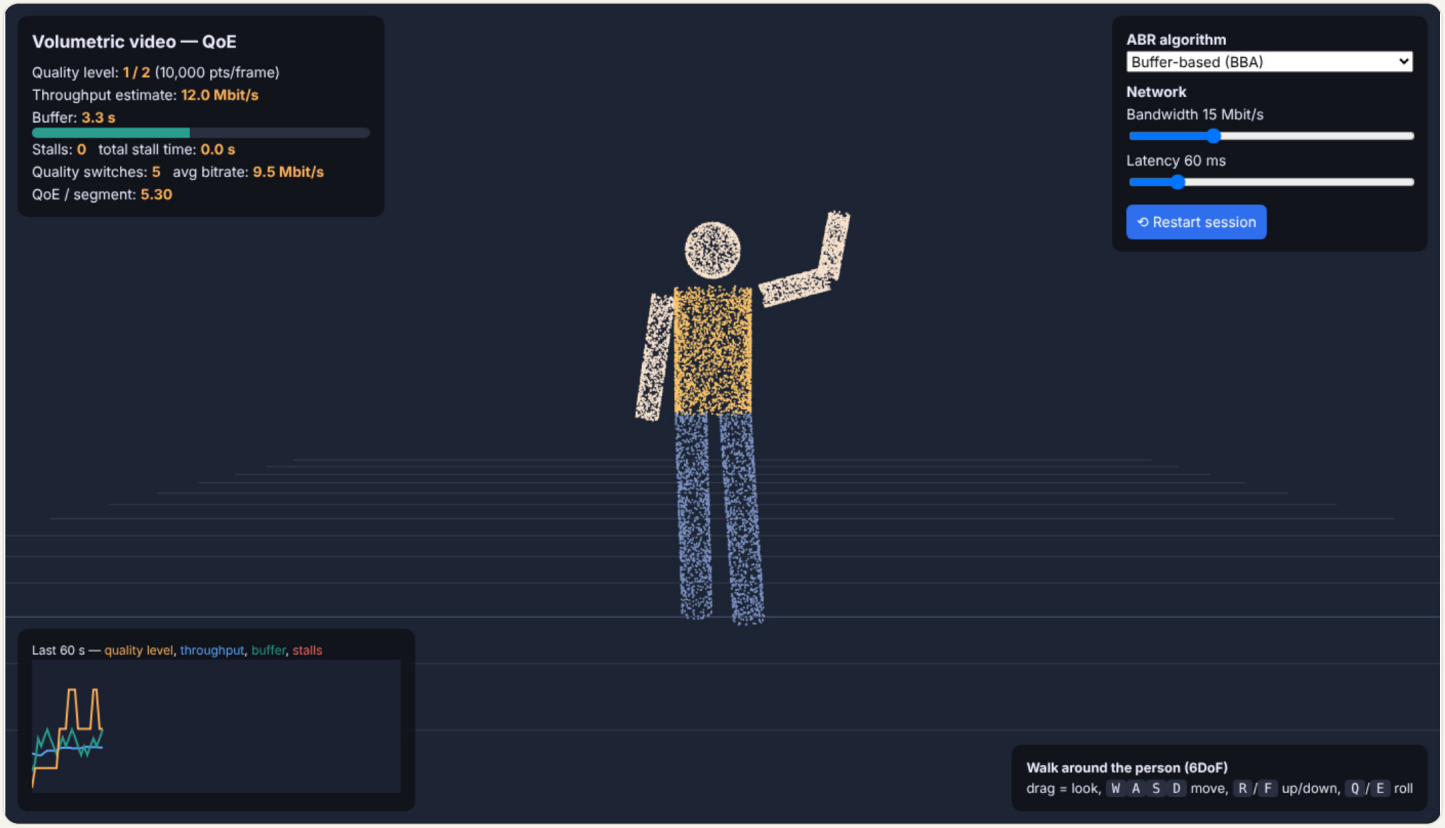
Predict where the viewer will be



At 300 ms, ‘no prediction’ is off by 39 cm and 6°; constant velocity cuts that to 8 cm and 4°.

LIVE DEMO

The volumetric video player



python vv_server.py → localhost:8001. Switch ABR, drag the bandwidth slider, walk around the person.

CAPSTONE

Build a 6DoF streaming experience

A. Static scene

Stream your own scan with the Day 15 system plus two improvements.

01

B. Volumetric video

Extend the player: compression, better ABR, view-dependent quality.

02

C. Research replication

Re-implement a simplified paper idea and evaluate it with our tools.

03

THE BIG IDEA

Spatial AI needs 3D worlds that travel.

Digital twins, telepresence, robots and AR glasses all need accurate 3D environments, delivered fast enough to feel real.

Three things to remember

1. Volumetric video = point cloud frames, encoded as a bitrate ladder of segments.
2. ABR chooses quality from throughput and buffer; QoE punishes stalls hardest.
3. 6DoF clients predict the pose and fetch ahead; better prediction means fewer gaps.

BEFORE NEXT SESSION

Capstone: demo next week, report one week later.